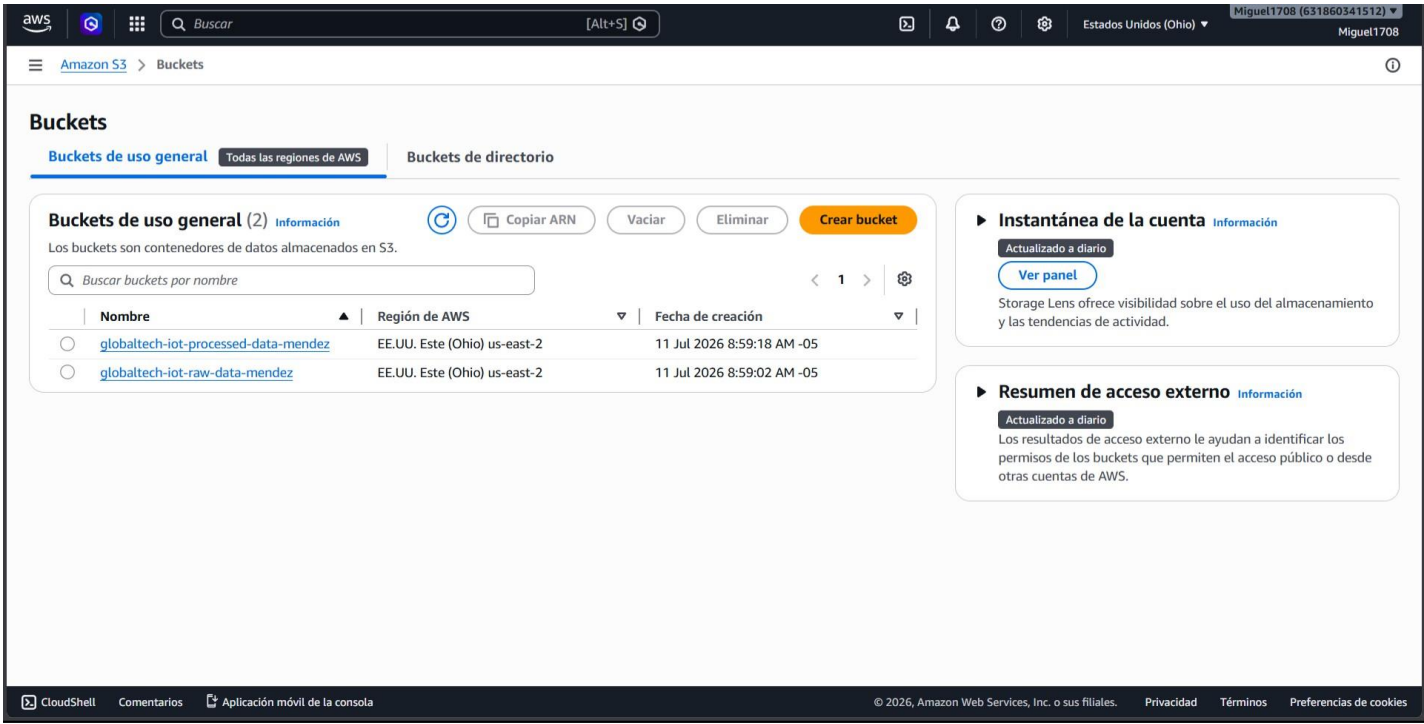
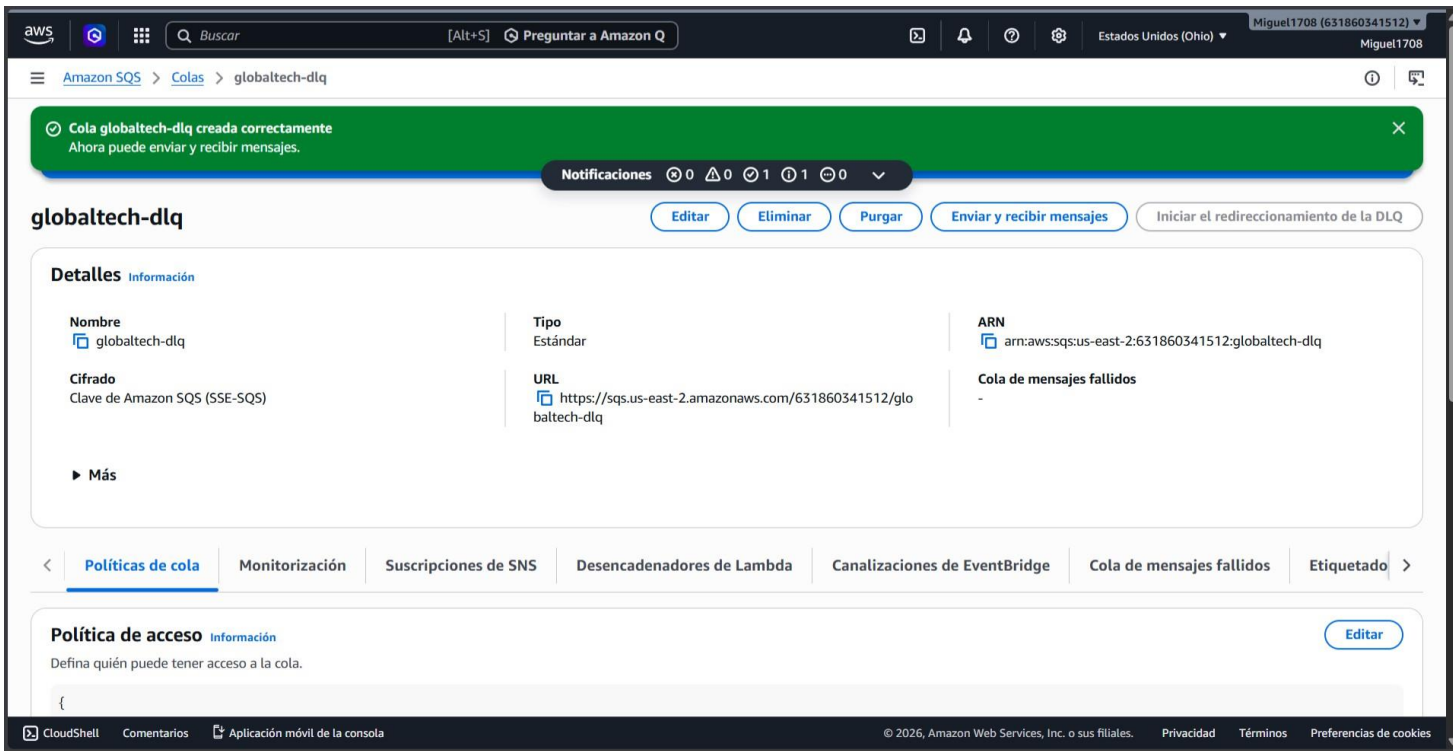


SOLUCIONARIO DEL EXAMEN FINAL – INFRAESTRUCTURA DE COMPUTACIÓN

1. Creación de buckets:



2. Crear las colas en Amazon SQS:



Cola de mensajes fallidos - Opcional [Información](#)

Envíe mensajes que no se pueden entregar a una cola de mensajes fallidos.

Establezca esta cola para recibir mensajes que no se puedan entregar.

- ☐ Deshabilitada
☒ Habilitada

Elegir cola

arn:aws:sqs:us-east-2:631860341512:globaltech-dlq

Recepciones máximas

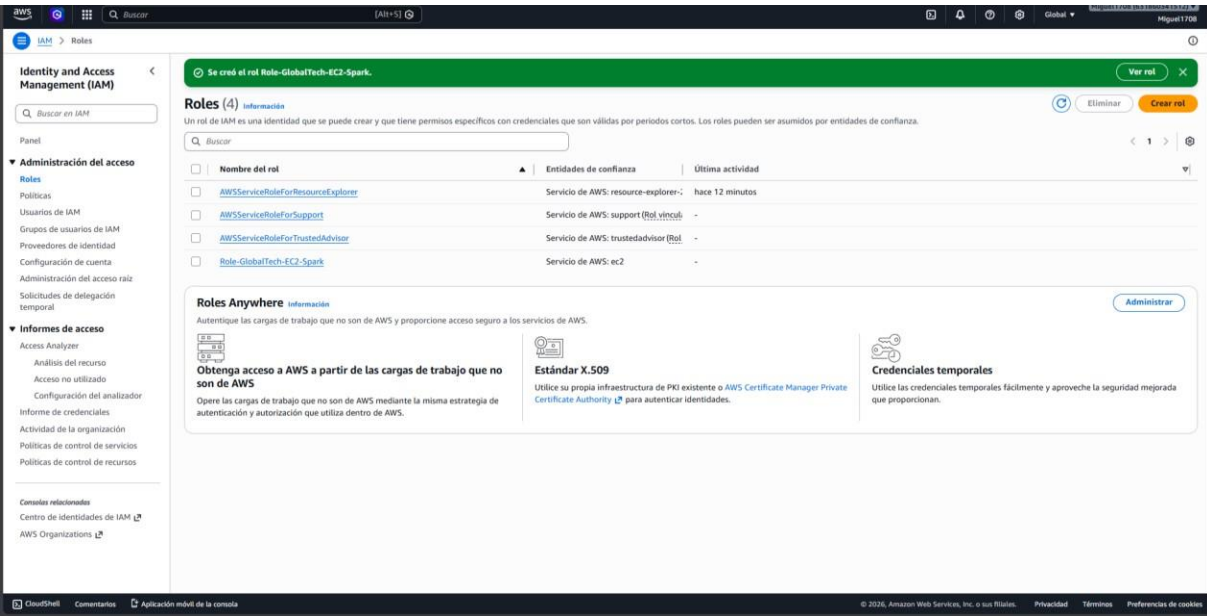
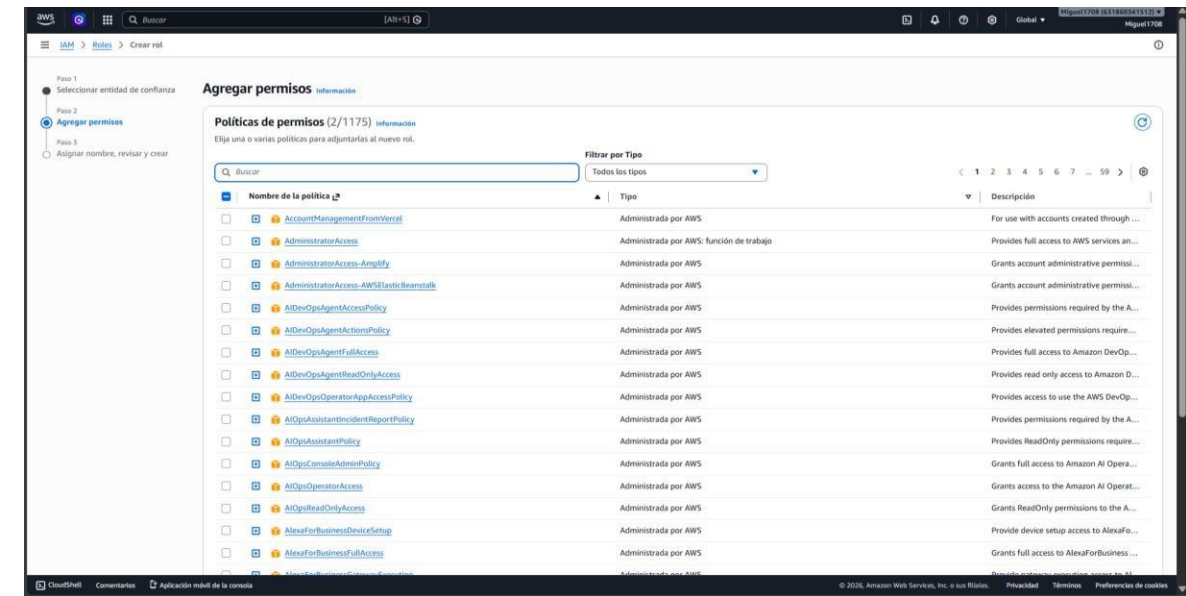
3

Debe estar entre 1 y 1000.

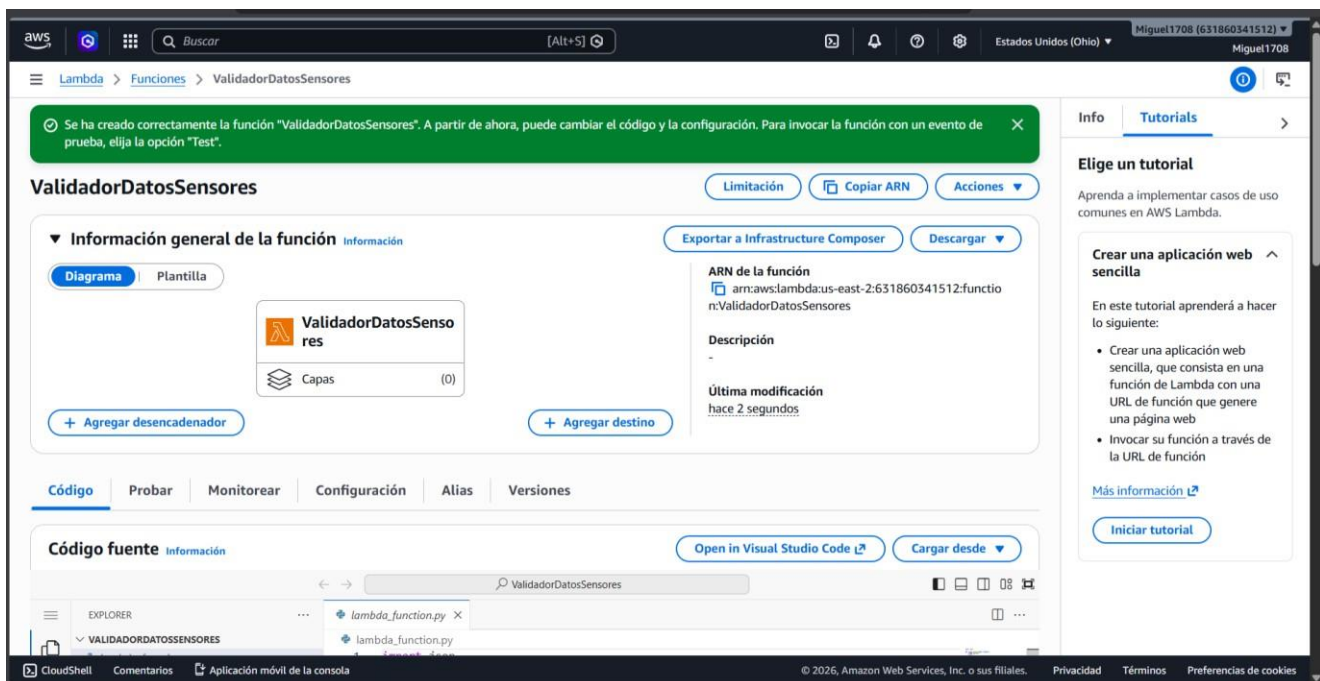
The screenshot shows the Amazon SQS console interface. At the top, a green notification bar states: "Cola globaltech-sensor-data-queue creada correctamente. Ahora puede enviar y recibir mensajes." Below this, the queue name "globaltech-sensor-data-queue" is displayed with buttons for "Editar", "Eliminar", "Purgar", "Enviar y recibir mensajes", and "Iniciar el redireccionamiento de la DLQ". The "Detalles" section shows the queue's name, type (Estándar), ARN, encryption status (Clave de Amazon SQS (SSE-SQS)), URL, and the status of the dead letter queue (Habilitada). A navigation bar at the bottom includes links for "Políticas de cola", "Monitorización", "Suscripciones de SNS", "Desencadenadores de Lambda", "Canalizaciones de EventBridge", "Cola de mensajes fallidos", and "Etiquetado".

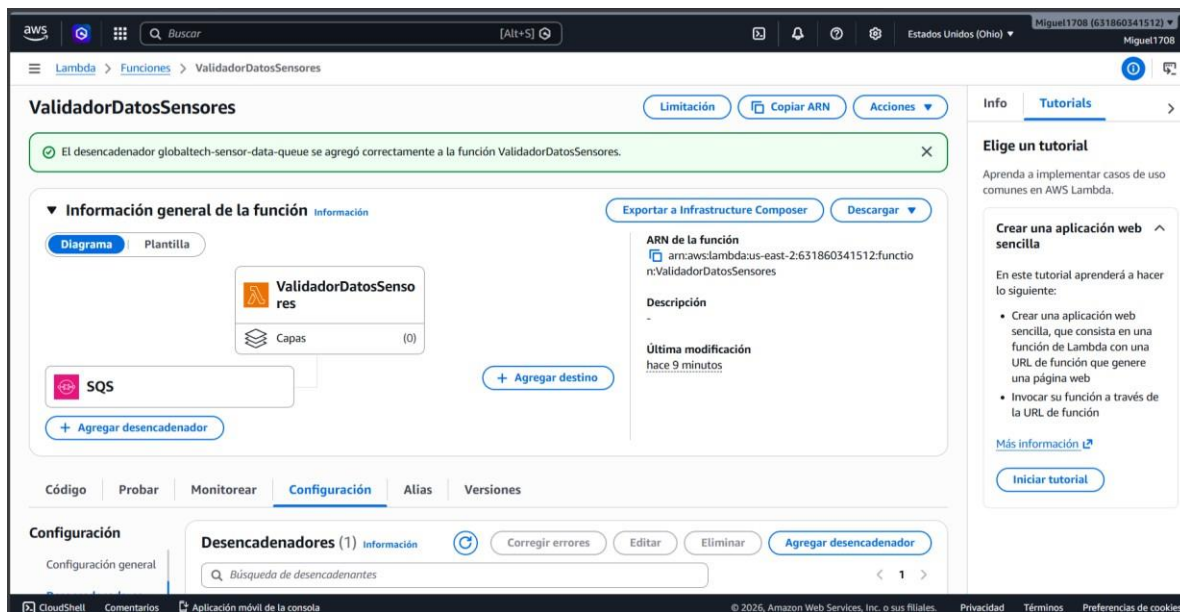
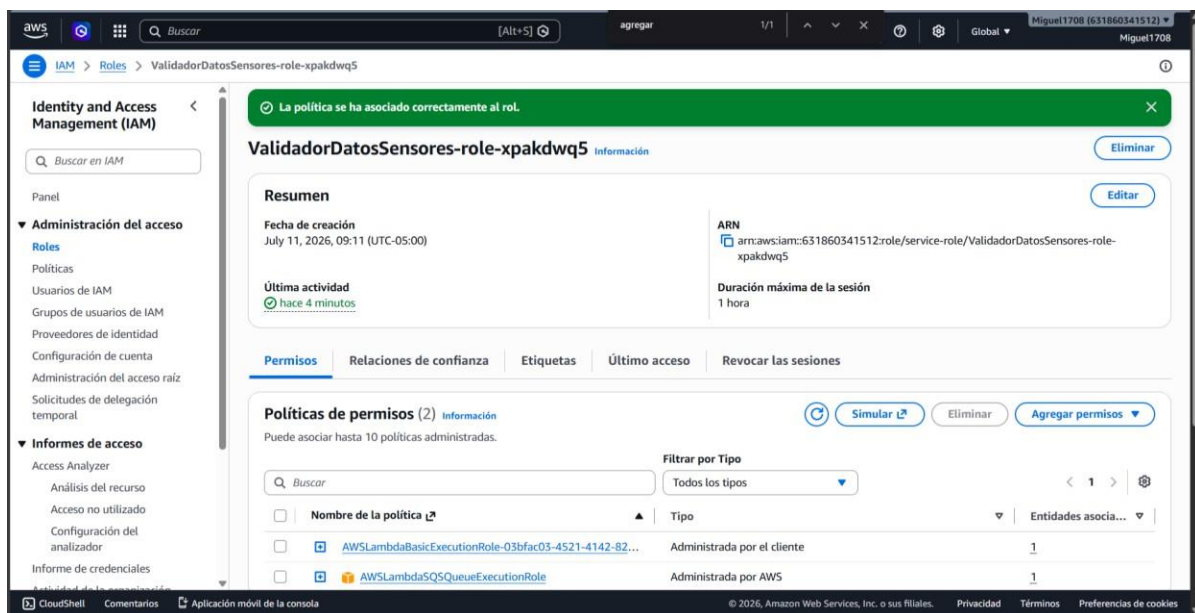
3. Configurar permisos con AWS IAM:

The screenshot shows the AWS IAM console's "Crear rol" (Create role) page. The "Tipo de entidad de confianza" (Trust entity type) is set to "Servicio de AWS" (AWS service). The "Caso de uso" (Use case) is set to "EC2". The "Política de confianza personalizada" (Custom trust policy) is selected. The "Caso de uso" (Use case) section shows the "EC2" role selected, which allows EC2 instances to call AWS services on their behalf. The "Política de confianza personalizada" (Custom trust policy) section shows the "EC2" role selected, which allows EC2 instances to call AWS services on their behalf.

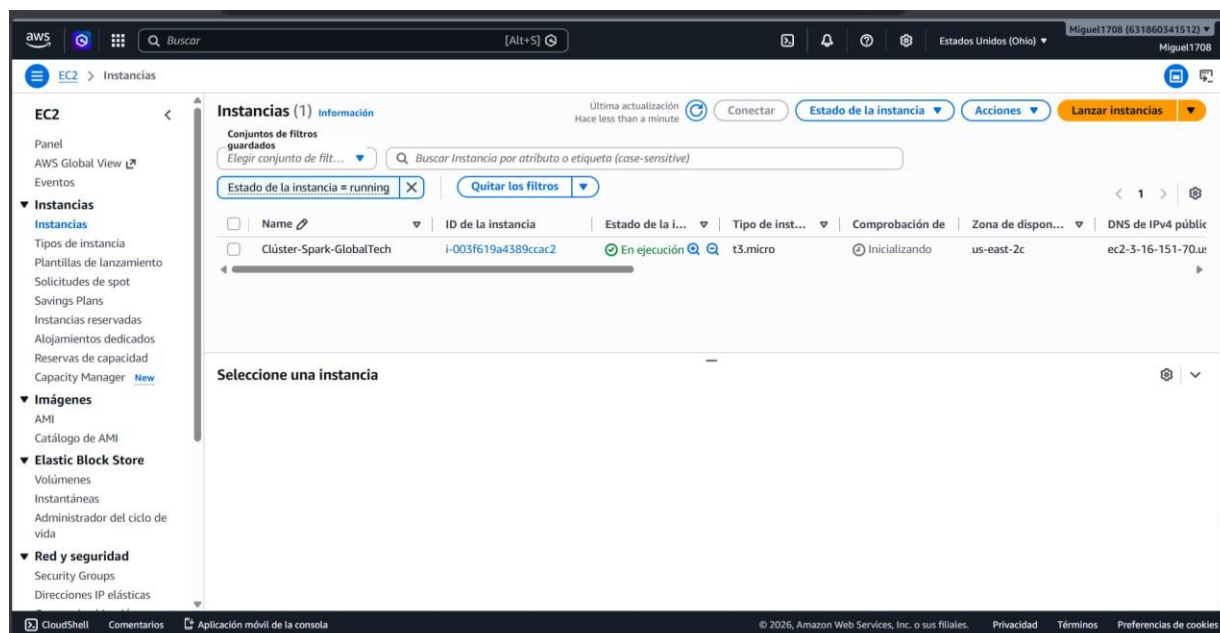


4. Crear función en AWS Lambda:





5. Lanzar el Servidor en Amazon EC2 (Para Apache Spark):



6. Crear la alarma en CloudWatch:

0.00423%

03:00 06:00 09:00 12:00

CPUUtilization

Nombre de la instancia
Clúster-Spark-GlobalTech

Estadística
Media

Período
15 minutos

Condiciones

Tipo de límite

☒ Estático
Utilice un valor como límite

☐ Detección de anomalías
Utilice una banda como límite

Cuando CPUUtilization sea...
Defina la condición de la alarma.

☒ Mayor
> límite

☐ Mayor/Igual
>= límite

☐ Menor/Igual
<= límite

☐ Menor
< límite

que...
Defina el valor del límite.
90
Debe ser un número.

7. Creacion de la alarma Alarma-Reinicio-Spark-CPU

CloudWatch > Alarmas

CloudWatch

Favoritos y recientes

Ingesta

Paneles

Alarmas

Operaciones de inteligencia artificial

Observabilidad de GenAI

Señales de aplicación (APM)

Monitorización de la infraestructura

Registros

Métricas

Se ha creado correctamente la alarma Alarma-Reinicio-Spark-CPU. [Ver alarma](#)

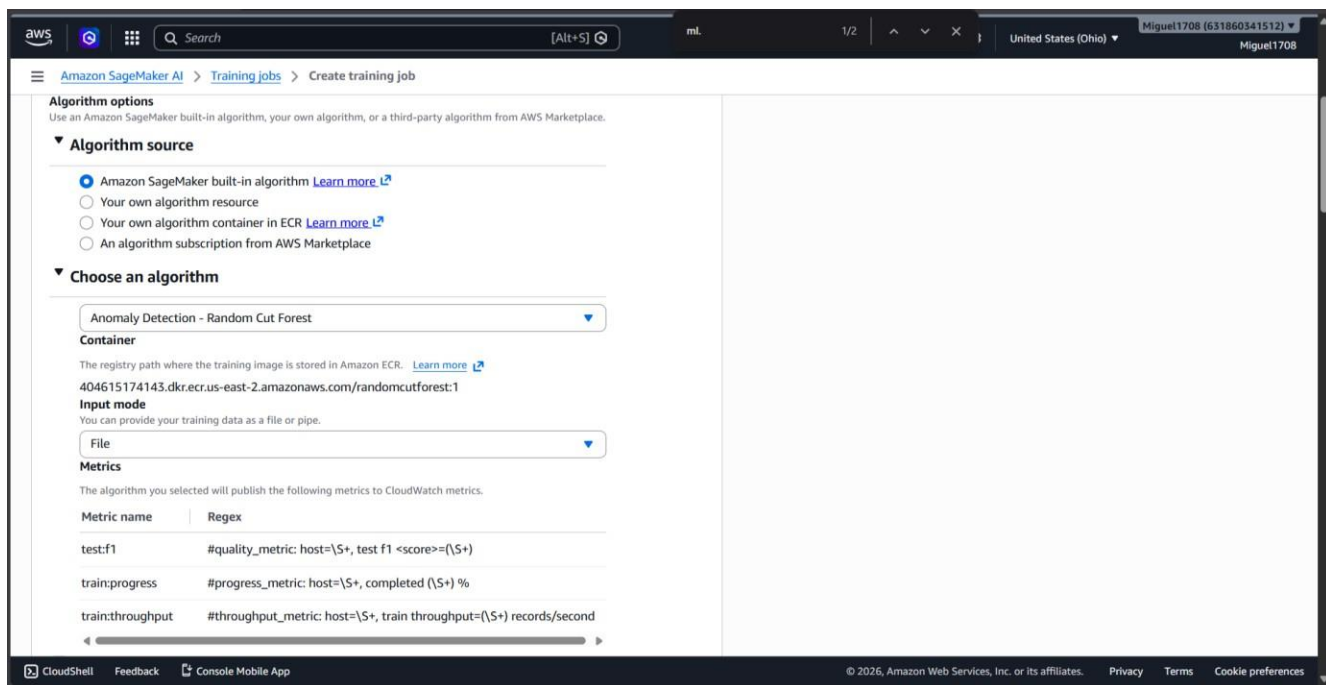
Alarmas Reglas del silencio - *nuevo*

Alarmas (1)

[Borrar selección](#) [Crear alarma compuesta](#) [Acciones](#) [Crear alarma](#)

[Filtros rápidos](#)

☐	Nombre	Estado	Acciones	Acciones silenciadas - nuevo	Última actualización del estado (UTC)	Condicio
☐	Alarma-Reinicio-Spark-CPU	Datos insuficiente	Acciones habilitadas	-	2026-07-11 14:38:10	CPUUtiliz dentro de



Solución a las preguntas técnicas propuestas:

- **¿Cómo configuraría el Batch Size y el Visibility Timeout de SQS para asegurar que la Lambda no procese mensajes duplicados ni pierda datos si el volumen aumenta repentinamente?**

Para manejar los picos de tráfico de los 50,000 sensores sin duplicar datos, pondría un Batch Size alto, de unos 100 a 500 mensajes por lote. Como son un montón de datos, procesarlos en grupos grandes reduce las llamadas a Lambda y ahorra costos. Para el Visibility Timeout, la regla de oro es que sea al menos 6 veces el tiempo que tarda la Lambda en ejecutarse. Si la Lambda tarda 10 segundos en validar los JSONs, le pondría unos 60 segundos a la cola. Así te aseguras de que mientras una Lambda revisa un lote, nadie más lo toque, evitando que se procesen mensajes repetidos si hay retrasos.

- **Si el proceso de Spark se queda sin espacio en disco durante una transformación compleja, ¿qué parámetros de EBS ajustaría y cómo optimizaría la configuración de Spark (spark.local.dir) para aprovechar el rendimiento del volumen?**

Si Spark se queda colgado y sin espacio haciendo un shuffle pesado, lo primero que haría sería entrar a la consola de AWS y ampliar los GiB del volumen gp3 sobre la marcha, que para eso son elásticos. También le subiría los IOPS y el rendimiento (Throughput) al disco para que lea y escriba más rápido. Del lado de Spark, el truco está en cambiar el parámetro spark.local.dir. Por defecto apunta a /tmp (que suele estar en el disco raíz y es chiquito), así que lo reconfiguraría para que apunte directamente a la ruta donde montamos el nuevo volumen gp3 de alta velocidad. Con eso el problema desaparece por completo.

- **Explique la diferencia entre usar un Estimator de SageMaker versus un contenedor propio (Bring Your Own Container) en términos de escalabilidad y gestión de dependencias.**

La diferencia clave es cuánto trabajo te quieres ahorrar. Con un Estimator, AWS te da todo masticado: te pone imágenes oficiales con XGBoost o Scikit-Learn ya listos y escala el entrenamiento a varias máquinas de forma nativa solo cambiando un número. En cambio, con un contenedor propio (BYOC), tú tienes el control total pero te toca hacer todo el trabajo sucio: armar el Dockerfile, instalar las librerías raras y programar a mano cómo se van a comunicar los nodos si usas más de un servidor. El Estimator es genial para ir rápido; BYOC es para cuando usas herramientas muy personalizadas.

- **Describe el flujo lógico para implementar un "Model Drift Detection" usando CloudWatch y que dispare un re-entrenamiento automático en SageMaker.**

El flujo es bastante lógico: primero activas SageMaker Model Monitor en tu endpoint para que guarde en S3 una muestra de los datos reales que van llegando y las predicciones que hace. Luego, el sistema corre una tarea por hora que compara esos datos nuevos con la "línea base" de tu entrenamiento original. Si detecta que el comportamiento de los datos cambió mucho, SageMaker le avisa a CloudWatch lanzando una métrica. Esto activa una alarma que manda un evento a EventBridge, y este finalmente dispara una Lambda o un pipeline para arrancar un re-entrenamiento automático con la data nueva en S3.

Pregunta de Complejidad Alta (3 puntos)

Suponga que por razones de costos, no puede mantener el clúster de EC2 encendido 24/7. ¿Cómo integraría AWS Lambda y CloudWatch Events (EventBridge) para levantar el clúster de Spark solo cuando haya suficientes datos en S3 y apagarlo automáticamente al finalizar la tarea, asegurando que los datos persistentes no se pierdan al terminar las instancias? (Pista: Hable sobre EBS persistentes vs. Efímeros y user data scripts).

Para automatizar el proceso, Amazon EventBridge detecta cuándo hay datos suficientes en S3 (o por horario) y activa una función AWS Lambda que enciende el clúster con `ec2.start_instances()`. Al arrancar, un script en el User Data de la EC2 inicia Spark de forma desatendida, ejecuta la transformación de datos y, justo al terminar de subir los archivos Parquet a S3, apaga automáticamente el servidor ejecutando un comando de detención (`aws ec2 stop-instances`).

Para evitar la pérdida de información entre ejecuciones, la base del sistema y sus configuraciones se guardan en volúmenes EBS persistentes con la propiedad `DeleteOnTermination = False`, asegurando que al pausar el cómputo (Stop) los datos no se borren. Por el contrario, los archivos temporales de Spark (`spark.local.dir`) se alojan en almacenamiento efímero (Instance Store) para aprovechar su máxima velocidad en los procesos de shuffle, ya que estos datos son volátiles y se pueden destruir sin problemas al apagar el equipo.

